Enhancing User Experience and Developer Workflow for NOS OFS Skill Assessment Package

Google Summer of Code 2026 (350 hours)

Tanmay Rajurkar
Vellore Institute of Technology – Andhra Pradesh
tanmayrajurkar1915@gmail.com
GitHub: tanmayrajurkar
Timezone: IST (UTC +5:30)

1. Personal Information

- **Name:** Tanmay Rajurkar
- **Email:** tanmayrajurkar1915@gmail.com
- **Country / Timezone:** India, IST (UTC +5:30)
- **GitHub:** https://github.com/tanmayrajurkar
- **Linkedin:** https://www.linkedin.com/in/tanmay-rajurkar-254305227/
- **University:** Vellore Institute of Technology, Andhra Pradesh
- **Program:** Integrated M.Tech (Computer Science)
- **Expected Graduation:** 2028

2. About Me

I am an undergraduate computer science student interested in applied scientific software engineering for operational data systems. My work focuses on improving the reliability, usability, and maintainability of research software used in real-world scientific environments.

I previously developed **SAGAR (Spatio-temporal Analytics Gateway for Aquatic Resources)** for the Centre for Marine Living Resources and Ecology (CMLRE), Ministry of Earth Sciences, Government of India. SAGAR is a marine data platform designed to integrate heterogeneous oceanographic, fisheries, and biological datasets into a unified analysis environment. The platform required structured ingestion pipelines capable of handling large scientific datasets such as NetCDF-based model outputs, automated quality control workflows aligned with IOOS/QARTOD principles, and visualization tools for scientific users.

Through this work, I gained practical experience building and maintaining data-driven scientific systems. My contributions included:

- Designing robust ingestion and processing pipelines for heterogeneous oceanographic datasets
- Implementing modular validation and quality control frameworks for scientific data
- Developing maintainable interfaces that simplify workflows for domain scientists
- Structuring testing and validation processes to support long-term system reliability

This experience aligns closely with the goals of the NOS OFS Skill Assessment project. It reinforced the importance of validation layers, reproducibility, and automated testing when developing operational scientific software.

Through this project, I aim to contribute improvements that strengthen developer confidence and usability of the system, particularly through expanded automated testing and CI infrastructure, improved GUI validation and usability, and clearer documentation for future contributors.

I am particularly interested in contributing to software that supports operational ocean forecasting systems, where correctness, transparency, and maintainability are just as important as the underlying scientific computations.

Abstract

The NOS OFS Skill Assessment package (`ofs-skill`, currently approaching the `v1.5.0-beta` release with a development milestone for `v1.6.0-beta`) is a Python-based framework used to evaluate operational forecast system (OFS) performance through observation retrieval, model processing, statistical skill computation, and visualization. Development follows a two-repository model: a developmental fork for active integration and experimentation, and an upstream repository for reviewed and stable releases. All proposed GSoC developments will target the developmental fork and be promoted through the existing review process to the upstream repository.

Recent changes have introduced modern package management using `pyproject.toml` and expanded pytest-based testing. The repository currently does not include an automated CI workflow where tests are executed systematically across environments. As the codebase grows, this makes it harder to introduce changes with confidence and increases the risk of regressions. In addition, although the existing codebase provides a `tkinter`-based GUI that integrates well with the backend pipeline, it currently offers limited input validation and minimal contextual guidance. This can create difficulties for users when configuring runs.

The focus of this project is to improve reliability, validation, and developer infrastructure without modifying the existing scientific computation pipeline. The primary objectives are to establish a GitHub Actions CI workflow with coverage checks, expand deterministic unit tests for core modules, and improve the GUI through clearer parameter grouping, stronger validation logic, and contextual tooltips derived from existing argument definitions.

In addition, the project will improve contributor onboarding by introducing a structured documentation system using `MkDocs`. This will allow the current long README file to be reorganized into clearer developer and user documentation.

As a secondary objective, and in collaboration with the team, a lightweight read-only results dashboard will be explored to provide a concise operational overview of OFS performance. The dashboard will focus on organizing and presenting outputs already generated by the skill assessment workflow and is intended to complement the parallel GIS application rather than duplicate scientific analysis functionality.

All proposed improvements will be incremental and backward compatible with the existing CLI workflow, improving usability, maintainability, and the long-term sustainability of the software.

3. Why This Project and IOOS?

My interest in this project is grounded in direct engagement with the codebase prior to drafting this proposal. I cloned the repository, configured the development environment from scratch, and executed the 1D skill assessment workflow through both the CLI and GUI in order to understand the full execution path, configuration structure, and output pipeline.

During this process, I identified and resolved a cross-platform GUI issue caused by the use of a Windows-only ttk theme. Replacing it with the cross-platform `'clam'` theme restored compatibility on macOS without affecting any scientific computation. The fix was submitted and merged (PR #6), demonstrating careful modification within the architectural boundaries of the project.

I subsequently contributed deterministic unit tests for a core utility function used in station–

node matching (PR #35), and submitted documentation improvements to clarify environment setup and contributor onboarding workflows (PR #32). The issue raised in PR #32 regarding environment configuration was later addressed in the repository through the introduction of a configuration file for API keys (PR #43), which incorporated the documentation clarification suggested in the original discussion.

What particularly draws me to this project is its operational context. The skill assessment framework is used to evaluate Operational Forecast Systems (OFS) developed and maintained by the National Oceanic and Atmospheric Administration (NOAA), particularly within NOAA CO-OPS and the broader IOOS ecosystem. In such operational environments, software reliability, configuration clarity, and reproducibility are critical because the results directly inform the evaluation of forecast systems used in real marine conditions.

Strengthening CI infrastructure, expanding deterministic test coverage, and improving validation logic therefore directly support the long-term stability and maintainability of these operational evaluation tools.

My previous work on SAGAR, a marine data platform developed for the Centre for Marine Living Resources and Ecology under the Ministry of Earth Sciences (India), further shaped my interest in IOOS-related systems. That project exposed me to many of the same challenges present in scientific software: heterogeneous datasets, reproducible workflows, and the need for sustainable engineering practices in research-driven systems.

This proposal builds on concrete contributions already merged or under review and focuses on strengthening infrastructure and usability without modifying the scientific computation pipeline. My goal is to contribute incrementally and collaboratively toward improving the long-term maintainability and contributor experience of the NOS OFS Skill Assessment framework.

3.1 Independent Marine Data Project: SAGAR

I developed **SAGAR (Spatio-temporal Analytics Gateway for Aquatic Resources)** in collaboration with researchers at the Centre for Marine Living Resources and Ecology (CMLRE), Ministry of Earth Sciences, Government of India.

SAGAR is a modular marine data platform designed to integrate heterogeneous oceanographic, fisheries, and biological datasets into a unified analysis environment. The system is distributed across multiple repositories under the T-MavLabs GitHub organization and spans data ingestion, processing, visualization, and analytics. The Python-based backend services are most relevant to this proposal:

- **Data processing engines** (DataProcessingEngine_SAGAR, data-processing) — Python/FastAPI pipelines responsible for ingesting and transforming heterogeneous oceanographic datasets across physical, biological, and fisheries data sources.
- **Quality control pipelines** informed by IOOS QARTOD principles, implemented within the data processing services to ensure data consistency and reliability.
- **Visualization and dashboard systems** (sagar-superset-brain, Sagar-Visual-Globe) — interactive dashboards and geospatial visualizations supporting exploration of marine datasets.

- **Configuration-driven execution workflows** enabling reproducible processing across the ingestion, transformation, and analytics pipeline.
- **Platform demonstration video**: Project Demo. (The public deployment is currently paused due to hosting limits.)

Developing SAGAR required addressing challenges very similar to those present in this project: designing structured validation layers for heterogeneous inputs, building modular Python services with clear separation between computation and presentation, and creating user-facing interfaces that lower the barrier to entry for domain scientists.

These experiences directly inform the improvements proposed in this project, particularly the GUI restructuring, CI integration, testing workflows, and the design of a lightweight dashboard for organizing and presenting results generated by the skill assessment pipeline.

3.2 Code References

- GUI compatibility fix (merged): PR #6
- Deterministic tests for station-node utilities: PR #35
- Documentation improvements and environment clarification: PR #32
- **IOOS QC example workflow notebook (merged):** Added a complete ERDDAP-to-QARTOD workflow example demonstrating integration of `erddapy` with `ioos_qc` using the PandasStream API, along with a reusable utility for summarizing QC flags and computing percentages. PR #190 (merged)

  This contribution involved multiple review iterations with maintainers, including refactoring logic for efficiency, aligning the notebook structure with existing examples, and following repository contribution standards. The final notebook demonstrates a full end-to-end workflow from ERDDAP data retrieval to QC evaluation and result summarization without modifying core library functionality.
- SAGAR Project Repository: Main SAGAR Repository

4. Contributor Evaluation Responses

As part of the contributor evaluation process for this project, I completed the tasks described in the project idea and documented the results.

Step 1: Execute a complete 1D skill assessment workflow Following the repository README, I successfully executed a full 1D skill assessment run for the Chesapeake Bay Operational Forecast System (CBOFS). This required:

- Cloning the repository and setting up the Conda development environment
- Configuring the `ofs_dps.conf` configuration file
- Downloading OFS model data using the provided scripts
- Running the skill assessment pipeline to generate plots and statistics

The run successfully generated the expected output directory structure (`data/model`, `data/observations`, `data/skill/stats`, and `data/visual`). The resulting statistics included RMSE, correlation, bias, and other NOS Standard Suite metrics, confirming that the entire pipeline executed correctly from data retrieval through visualization.

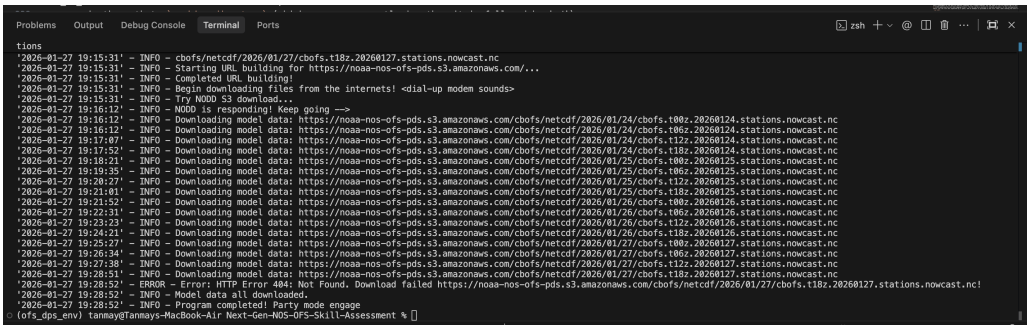

Figure 1: Model data download process for OFS datasets.

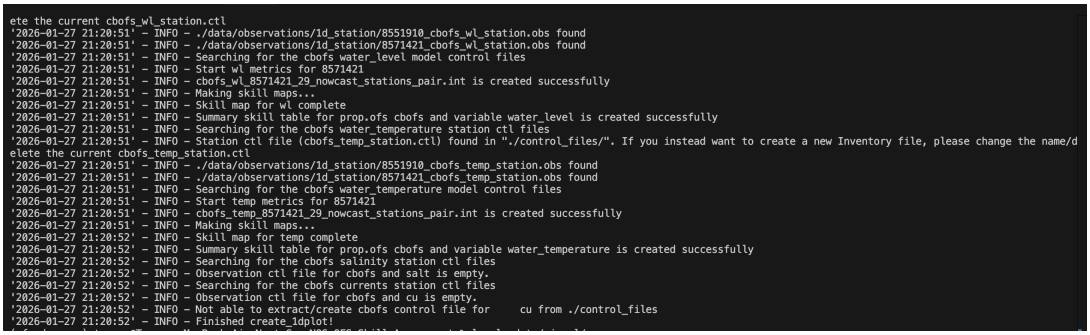

Figure 2: Example output from the 1D skill assessment workflow showing time series comparison and skill statistics.

Step 2: Execute a second run with modified configuration To further understand the workflow and configuration options, I completed a second run using different input parameters. This run included:

- Multiple observation providers (`-so co-ops,ndbc`)
- Multiple variables (`-vs water_level,water_temperature`)

The software correctly generated plots and statistics for both variables across the expanded station set. These experiments helped confirm that the pipeline correctly handles multiple station providers, multiple variables, and automated generation of output artifacts.

Step 3: Implement deterministic unit tests To strengthen regression protection within the repository, I implemented a pytest-based unit test for a deterministic utility function used in station–node matching within the `model_processing` module.

This contribution was submitted as:

- PR #35

Step 4: Improve the GUI interface While exploring the GUI workflow, I contributed improvements to the `create_gui.py` interface. In particular:

- Added contextual tooltips to GUI input parameters
- Improved usability by providing inline explanations for command line flags within the interface

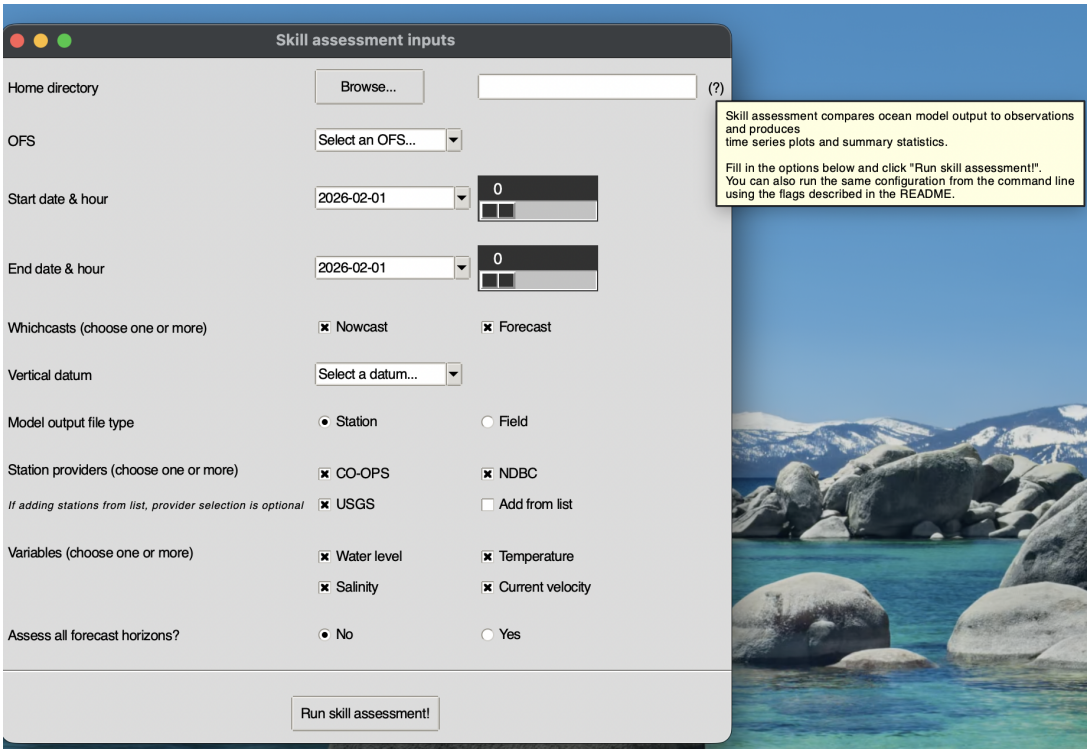

Figure 3: Improved GUI interface showing contextual tooltips for configuration parameters.

Additional repository contributions During the contributor evaluation process I also submitted several additional improvements:

- **Cross-platform GUI compatibility fix** Replacing a Windows-only `ttk` theme with the cross-platform `clam` theme to ensure the GUI works correctly on macOS and Linux. PR #6 - Merged
- **Documentation improvements for environment setup** Clarified development environment configuration and contributor onboarding. PR #32

  The issue discussed in PR #32 was later addressed in the repository through configuration-based API key loading: PR #43

These evaluation tasks helped me understand the full architecture of the repository, including the CLI entry points, configuration system, data-processing pipeline, and visualization components. The experience gained during this phase directly informed the technical approach proposed in the following sections.

5. Project Details

5.1 Project idea link -

Enhance NOS skill assessment package's user and developer experience

5.2 Technical Description

The NOS OFS Skill Assessment package is an installable Python package organized under `src/ofs_skill/`, with CLI entry points located in `bin/`. Execution follows a parameter-driven workflow in which arguments are collected through `argparse` (or via the tkinter GUI when required arguments are not provided), merged with configuration values from `conf/ofs_dps.conf`, and passed into a shared backend pipeline centered on the `ModelProperties` object.

This architecture enables a clear separation between configuration, data retrieval, model processing, skill computation, and visualization, supporting reproducible evaluation of NOAA Operational Forecast Systems (OFS).

Architectural Boundaries. The computational pipeline is organized into four core subpackages:

- **Model Processing** — handling OFS model files, forecast cycle logic, datum offsets, and station–node matching.
- **Observation Retrieval** — querying station inventories (CO-OPS, NDBC, USGS) and retrieving observation data for comparison with model outputs.
- **Skill Assessment** — constructing paired time-series and computing NOS Standard Suite metrics.
- **Visualization** — generating 1D and 2D plots and providing the tkinter-based GUI interface.

The GUI collects parameters and returns a `SimpleNamespace` object that is consumed identically by the backend pipeline. This separation between argument collection and scientific computation is a core design strength of the system and will remain unchanged. Scientific metric definitions, pairing logic, and computational algorithms will not be modified as part of this project.

Priority Work: Infrastructure and Reliability Improvements.

i. Continuous Integration

A GitHub Actions workflow will be introduced in the developmental fork to automatically validate changes across supported environments.

The workflow will:

- Create the development environment using the existing `environment.yml`.
- Execute `pytest` across a Python version matrix (Python 3.9–3.12).

- Enforce coverage thresholds using `pytest-cov`.
- Exclude network-dependent tests by default using `-m "not network"`.
- Run pre-commit checks (ruff, mypy, pyupgrade) to maintain code quality.

After the CI pipeline is validated, `branch protection rules` can be introduced to require successful CI checks before pull requests are merged. This ensures that all contributions are automatically tested and that regressions cannot be introduced into the codebase without passing the established validation workflow.

Once stabilized in the developmental fork, the CI configuration and associated policies can be promoted to the upstream repository.

ii. Testing Expansion

Test development will prioritize deterministic functions that perform core computational logic without requiring external network access, ensuring fast and reliable automated testing.

External API calls (CO-OPS, NDBC, USGS, and AWS S3) will be mocked to guarantee offline execution of tests. Integration tests requiring network access will remain optional and isolated using the existing `network` pytest marker.

Test coverage (currently approximately 16%) will be increased incrementally from the established baseline.

Tests will be organized across multiple tiers to improve maintainability:

- Unit tests for deterministic computational functions
- Integration tests for module-level interactions
- End-to-end workflow tests validating complete skill assessment runs

iii. GUI Enhancements

Improvements will be localized to `create_gui.py`, preserving the existing interface and downstream execution behavior. Planned updates include:

- Logical parameter grouping using `ttk.LabelFrame` to improve clarity of configuration inputs.
- Contextual tooltips derived directly from existing `argparse` help strings to avoid documentation duplication.
- Consolidated validation logic enforcing:
  - Non-empty and writable output directory paths,
  - Correct date ordering (start-date < end-date),
  - Detection of incompatible option configurations, with validation rules finalized in consultation with the development team to ensure alignment with existing workflow constraints.

These validation checks will be implemented as standalone helper functions, allowing them to remain reusable and testable without altering the existing CLI behavior. GUI validation will therefore act as an early safeguard rather than replacing backend validation.

Secondary Work:

i. Structured Documentation using MkDocs

To improve developer onboarding and documentation organization, a structured documentation system using **MkDocs** will be introduced.

The current repository relies heavily on a very long README file that contains installation instructions, usage guidance, and development information. While this information is useful, its current format can make navigation difficult for new contributors.

The proposed MkDocs documentation site will:

- Organize installation, usage, and development guides into structured documentation pages.
- Provide clearer onboarding instructions for new contributors.
- Include architectural explanations and workflow diagrams for the major subpackages.
- Automatically generate API documentation where appropriate.

This documentation system will complement the existing README while making the project easier to navigate for developers and scientific users.

ii. Results Dashboard (Complementary Layer)

A lightweight, read-only dashboard implemented under `ofs_skill/visualization/dashboard.py` will provide a concise operational overview of completed skill assessment runs.

The dashboard will:

- Detect completed runs by scanning existing output directories,
- Parse generated plots and summary statistics,
- Present results through an interactive interface,
- Avoid recomputation of skill metrics.

Because all outputs are already generated by the existing workflow, the dashboard will focus on organizing and summarizing these results rather than performing additional analysis.

During discussions with the mentoring team, it was noted that the project is also exploring a separate GIS-based visualization system. Therefore, the scope and structure of this dashboard will be refined in collaboration with mentors to ensure it complements the GIS application and provides a useful operational overview of OFS performance.

5.3 Community Bonding Period

The community bonding period will focus on scope confirmation, infrastructure validation, and alignment with mentors prior to beginning implementation.

i. Architectural Re-Familiarization. Although I have already executed both 1D and 2D workflows during contributor evaluation, I will conduct a structured walkthrough of the full execution path (CLI and GUI) to document module interactions and confirm boundaries between argument handling, backend computation, and visualization. This will refine the list of deterministic test targets identified in the proposal and ensure a clear understanding of the existing architecture before implementation begins.

ii. Priority and Scope Alignment. In coordination with mentors, I will finalize:

- The prioritization of CI infrastructure, testing expansion, and GUI improvements.
- The effective Python support range for CI.
- Initial coverage baselines and milestone-based threshold targets.
- The expected scope of the post-run dashboard to ensure it complements the parallel GIS application without expanding beyond a read-only viewer.

This discussion will ensure the coding period begins with clearly defined deliverables and no ambiguity regarding architectural boundaries.

iii. CI Preparation and Coverage Baseline. I will measure the current test coverage using the existing `pytest-cov` configuration and document the baseline. A draft CI workflow will be prepared in a feature branch to validate dependency resolution and platform compatibility (the GitHub Actions Linux runner), while the formal CI integration will be implemented during the coding period.

iv. GUI Validation Planning. I will outline the proposed validation logic structure and parameter grouping strategy for early mentor feedback before modifying the existing GUI implementation. This will ensure the validation rules and configuration constraints align with the expectations of the development team.

v. Communication Plan. Weekly written updates will summarize progress, blockers, and next steps. Technical discussions will primarily occur through GitHub issues and pull requests. In addition, I will participate in the project's **weekly team meetings** to stay aligned with ongoing development and collaborate with the broader team. If I am unable to attend a meeting due to timezone differences, I will provide my **weekly updates in the meeting slides** as suggested.

Additional discussions can be scheduled via Google Meet when deeper technical conversations are needed. I will maintain consistent overlap with the U.S. Eastern timezone to ensure smooth communication with the NOAA and IOOS teams.

By the end of community bonding, priorities will be finalized, coverage baselines established, and implementation plans confirmed, allowing the coding phase to proceed with a stable and agreed-upon scope.

6. Schedule of Deliverables

The project follows the standard GSoC 2026 timeline for a 12-week coding period. All work will be developed incrementally in the developmental fork and merged through focused pull requests. The scientific computation pipeline will remain unchanged.

Community Bonding Period (May 1 – May 24, 2026)

- Confirm project priorities with mentors.
- Establish baseline test coverage.
- Review repository architecture and execution workflow.
- Prepare CI workflow draft and verify environment reproducibility.
- Finalize validation rules for GUI configuration options.

Phase 1 (Weeks 1–6: May 25 – July 6)

Weeks 1–2 (May 25 – June 7): Continuous Integration

- Implement GitHub Actions CI workflow.
- Configure environment setup using `environment.yml`.
- Run `pytest` across Python version matrix (3.9–3.12).
- Introduce coverage checks using `pytest-cov`.

Weeks 3–4 (June 8 – June 21): Deterministic Test Expansion

- Expand unit tests for core computational utilities.
- Mock external API calls for offline test execution.
- Increase coverage thresholds incrementally.

Weeks 5–6 (June 22 – July 6): GUI Improvements

- Implement GUI validation logic (dates, directories, configuration compatibility).
- Refactor validation logic into reusable helper functions.
- Introduce parameter grouping using `ttk.LabelFrame`.

Midterm Evaluation (Around July 10, 2026)

- CI fully operational.
- Significant improvement in deterministic test coverage.
- GUI validation and usability improvements implemented.

Phase 2 (Weeks 7–12: July 7 – August 17)

Weeks 7–9 (July 7 – July 27): Results Dashboard

- Implement read-only dashboard for completed runs.
- Parse existing output directories and render stored results.
- Provide filtering by OFS, variable, station, and whichcast.

Weeks 10–11 (July 28 – August 10): Testing and Documentation

- Add integration tests using fixture output directories.
- Expand edge-case coverage for skill metric utilities.
- Introduce MkDocs documentation structure.

Week 12 (August 11 – August 17): Final Refinement

- Finalize documentation and contributor guides.
- Improve docstrings in modified modules.
- Validate reproducibility from clean installation.

Final Week (August 17 – August 24)

- Submit final project deliverables.
- Coordinate promotion of changes to the upstream repository.
- Submit final GSoC evaluation and project report.

7. Availability and Commitments

I will be available **full-time during the GSoC 2026 coding period** (May 25 – August 17) and can dedicate approximately **35–40 hours per week** to the project. My typical work schedule will consist of focused development blocks (4–6 hours of deep work per day), along with additional time allocated for testing, documentation, and communication with mentors.

I am currently an undergraduate student at Vellore Institute of Technology – Andhra Pradesh. During the core GSoC coding period I do not anticipate any major academic examinations, internships, or external commitments that would significantly reduce my availability. If minor academic obligations arise (such as internal assessments), they are short in duration and will not affect weekly development milestones.

To manage both academic responsibilities and project work effectively, I plan to:

- Allocate fixed daily hours dedicated exclusively to GSoC development.
- Maintain **consistent and proactive communication** with mentors regarding progress, blockers, and milestones.
- Deliver work incrementally through **small, reviewable pull requests** to ensure continuous feedback and integration.

The mentoring team primarily operates in the U.S. Eastern timezone. I will maintain **regular overlap with this timezone** to participate in scheduled discussions and collaborative work.

I will actively participate in the project's weekly team meetings to stay aligned with ongoing development and expectations. If I am unable to attend due to timezone or scheduling constraints, I will provide structured weekly updates in the

shared meeting slides to ensure clear visibility of progress and maintain alignment with the team.

My goal during the coding period is to maintain **steady, transparent progress** through well-structured contributions, clear documentation, and consistent collaboration with the project mentors and contributors.

8. AI Disclosure

Use of AI in Proposal Development

During preparation of this proposal, AI tools were used only for *AI-assisted copy editing*. This included improving grammar, sentence clarity, and formatting of human-written text. **The core ideas, technical design decisions, architecture analysis, and project plan were developed independently based on my direct exploration of the repository, local execution of the workflow, and discussions with the project mentors.**

AI assistance was therefore limited to refining readability and presentation rather than generating the proposal content itself.

Use of AI in Code or Technical Work

AI tools were occasionally used as a reference tool while exploring the repository and preparing implementation ideas, similar to consulting documentation or development forums. Any technical understanding, design decisions, and proposed implementations in this proposal were verified through direct examination of the repository codebase and local testing of the system.

All code contributions referenced in this proposal (e.g., PR #6 and test additions in the development repository) were written, reviewed, and tested manually to ensure full understanding of the implementation and its impact on the existing architecture.

9. Appendix: Design Overview

This appendix summarizes the current repository architecture and the architectural boundaries respected by the proposed improvements. All descriptions reflect the existing implementation in `NOAA-CO-OPS/Next-Gen-NOS-OFS-Skill-Assessment`, which supports evaluation workflows for NOAA Operational Forecast Systems (OFS).

The purpose of this appendix is to illustrate how the proposed improvements integrate with the existing architecture while preserving the scientific computation pipeline.

9.1 System Architecture Overview

The package follows a layered architecture. User-facing entry points reside in `bin/` and accept parameters through `argparse`. A central INI configuration file (`conf/ofs_dps.conf`) provides directory paths and runtime settings. Additional configuration such as API keys is now supported through a separate configuration file loaded during runtime.

The core library under `src/ofs_skill/` is organized into four subpackages:

- `model_processing` — OFS file handling, forecast cycle logic, node matching, and datum offsets.

- `obs_retrieval` — station inventories, observation download, formatting, and control file generation.
- `skill_assessment` — paired time-series construction and NOS Standard Suite metrics.
- `visualization` — 1D/2D plotting utilities and the tkinter GUI.

The GUI (`create_gui.py`) receives an `argparse.ArgumentParser`, builds a tkinter form, and returns a `SimpleNamespace` identical to the CLI namespace. This separation between argument collection and computation is a design strength and will remain unchanged.

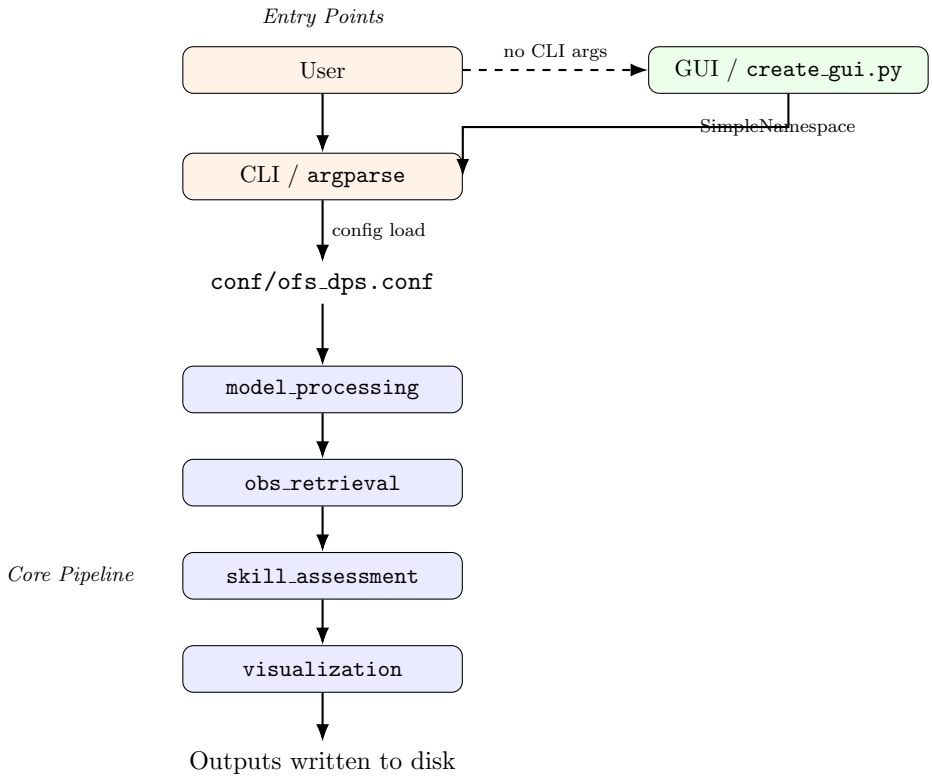

Figure 4: System Architecture Diagram

9.2 Execution Flow (1D Skill Assessment)

The primary entry point is `bin/visualization/create_1dplot.py`. The high-level execution flow is:

1. Parse CLI arguments via `argparse`.
2. If required arguments are missing, launch GUI and return populated namespace.
3. Populate a `ModelProperties` object.
4. Execute `create_1dplot(prop)`:
   (a) Read configuration file.
   (b) Validate parameters.
   (c) Create output directory tree.

(d) Iterate over selected variables to generate control files, paired datasets, and plots.

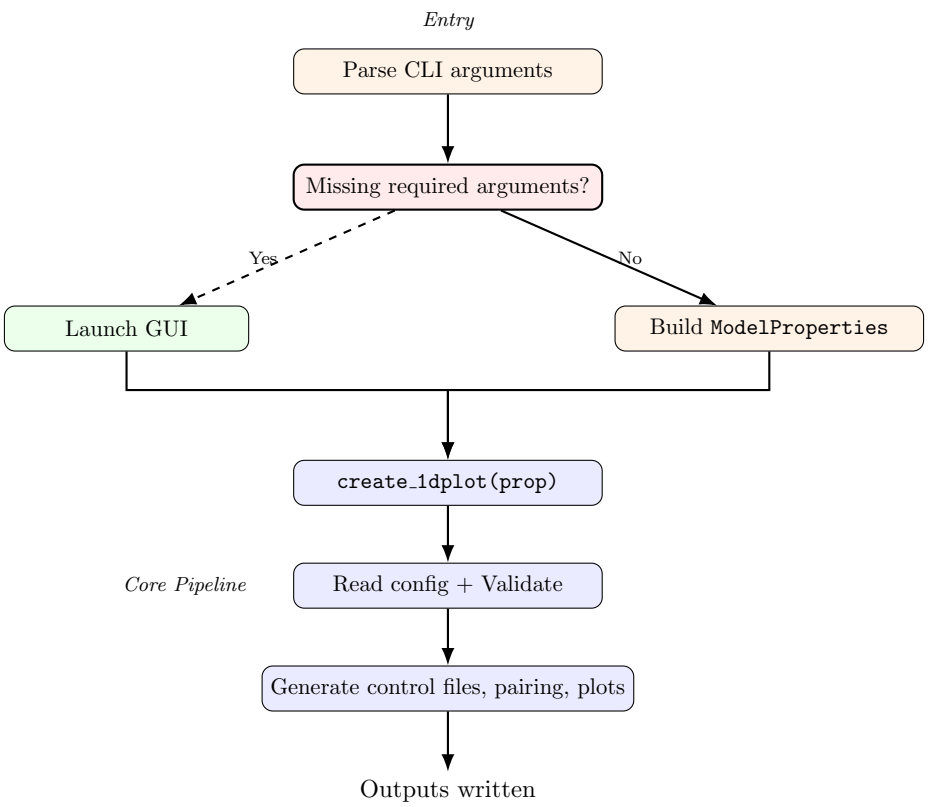

Figure 5: 1D Skill Assessment Execution Flow

9.3 GUI Enhancement Design

Current state. The GUI constructs a flat grid of widgets and performs basic required-field checks. It does not enforce date ordering, path writability, or logical option compatibility.

Improved state.

- Structured validation (date ordering, directory checks, compatible options).
- Full tooltip coverage derived from existing `argparse` help text.
- Logical grouping using `ttk.LabelFrame`.
- No modification of backend interfaces or computation logic.

9.4 Documentation Architecture

The repository currently relies heavily on a long README file that contains installation instructions, usage guidance, and development notes.

To improve developer onboarding and documentation organization, structured documentation using **MkDocs** will be introduced. The documentation site will:

- Organize installation, usage, and development guides into structured pages.

- Provide architectural explanations and workflow diagrams.
- Document contributor workflows and development practices.

This documentation system will complement the existing README while making the project easier to navigate for new contributors.

9.5 Dashboard Integration Design

The dashboard is a read-only post-run viewer that operates independently of the computational pipeline.

Data sources.

- Plots in `data/visual/`
- Skill statistics in `data/skill/stats/`
- Control files and paired outputs

Behavior.

- Launched via `python -m ofs_skill.dashboard <output_dir>`.
- Discovers artifacts from completed runs.
- Provides filtering by OFS, variable, station, and whichcast.
- Performs no recomputation and writes no files.

During discussions with the mentoring team it was noted that a GIS-based visualization platform is also being explored. The proposed dashboard therefore focuses only on organizing outputs from the skill assessment pipeline and presenting them in a concise operational view rather than duplicating analytical capabilities of the GIS system.

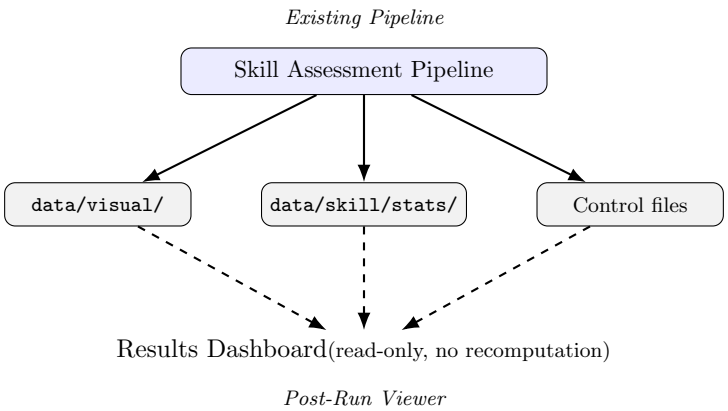

Figure 6: Dashboard Integration

9.6 Testing and CI Architecture

Current state. A limited pytest foundation exists, but broader subpackage coverage is incomplete and no CI workflow currently runs automatically.

Proposed architecture.

- Deterministic unit tests across core subpackages.
- Mocked external dependencies for offline CI reliability.
- Incremental coverage thresholds relative to measured baseline.
- GitHub Actions workflow enforcing automated testing.
- Branch protection rules requiring successful CI checks before merging.

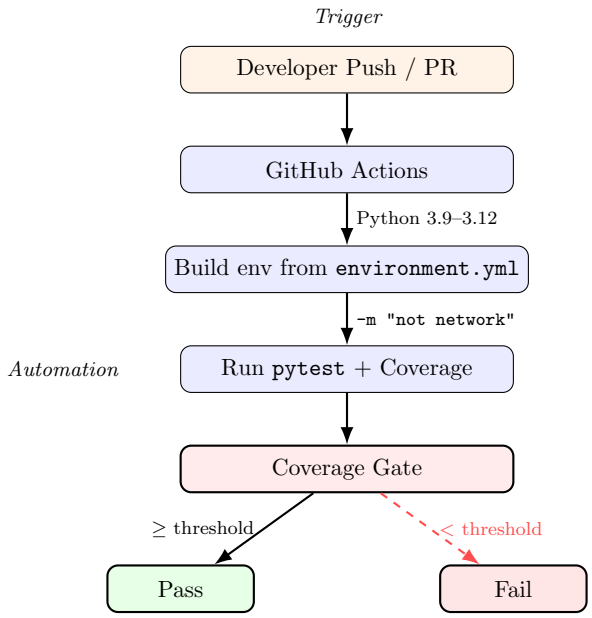

Figure 7: CI Pipeline Architecture